

Untitled5.ipynb_

[1]

```
import pandas as pd
```

[2]

```
data = {  
  
    "Study_Hours": [1,2,3,4,5,6,7,8,9,10,11,12],  
  
    "Attendance": [60,65,70,72,75,80,82,85,88,90,92,95],  
  
    "Assignments": [2,3,4,5,5,6,7,7,8,8,9,10],  
  
    "Marks": [35,40,45,50,58,65,72,78,82,88,92,96]  
  
}
```

```
df = pd.DataFrame(data)
```

[3]

```
df.to_csv("student_performance.csv", index=False)
```

[4]

```
df = pd.read_csv("student_performance.csv")
```

```
[5]
```

```
0s
```

```
print("First 5 Rows:")
```

```
print(df.head())
```

First 5 Rows:

	Study_Hours	Attendance	Assignments	Marks
0	1	60	2	35
1	2	65	3	40
2	3	70	4	45
3	4	72	5	50
4	5	75	5	58

```
[6]
```

```
0s
```

```
print("Last 5 Rows:")
```

```
print(df.tail())
```

Last 5 Rows:

	Study_Hours	Attendance	Assignments	Marks
7	8	85	7	78
8	9	88	8	82
9	10	90	8	88
10	11	92	9	92
11	12	95	10	96

```
[7]
```

```
0s
```

```
print("Dataset Shape:")
```

```
print(df.shape)
```

Dataset Shape:
(12, 4)

[8]

0s

```
print("Column Names:")
```

```
print(df.columns)
```

Column Names:
Index(['Study_Hours', 'Attendance', 'Assignments', 'Marks'],
dtype='object')

[9]

0s

```
print("Data Types:")
```

```
print(df.dtypes)
```

Data Types:
Study_Hours int64
Attendance int64
Assignments int64
Marks int64
dtype: object

[10]

0s

```
print("Summary Statistics:")
```

```
print(df.describe())
```

Summary Statistics:

	Study_Hours	Attendance	Assignments	Marks
count	12.000000	12.000000	12.000000	12.000000
mean	6.500000	79.500000	6.166667	66.750000
std	3.605551	11.172206	2.443296	21.080474
min	1.000000	60.000000	2.000000	35.000000
25%	3.750000	71.500000	4.750000	48.750000
50%	6.500000	81.000000	6.500000	68.500000
75%	9.250000	88.500000	8.000000	83.500000
max	12.000000	95.000000	10.000000	96.000000

Part 1 Conclusion:

The dataset contains 12 rows and 4 columns.

The available features are:

1. Study_Hours
2. Attendance
3. Assignments

The target variable is: Marks

All columns are numerical (int64) and there are no missing values in the dataset.

The dataset is clean and ready for machine learning model training and analysis.

```
[11]
```

```
X = df[['Study_Hours', 'Attendance', 'Assignments']]
```

```
y = df['Marks']
```

```
[12]
```

```
0s
```

```
print("Features (X):")
```

```
print(X)
```

```
Features (X):
```

	Study_Hours	Attendance	Assignments
0	1	60	2
1	2	65	3
2	3	70	4
3	4	72	5
4	5	75	5
5	6	80	6
6	7	82	7
7	8	85	7
8	9	88	8
9	10	90	8
10	11	92	9
11	12	95	10

```
[13]
```

```
0s
```

```
print("\nLabel (y):")
```

```
print(y)
```

```
Label (y):
```

```
0      35
```

```
1      40
2      45
3      50
4      58
5      65
6      72
7      78
8      82
9      88
10     92
11     96
Name: Marks, dtype: int64
```

Features:

1. Study_Hours
2. Attendance
3. Assignments

Label: Marks

Explanation: Features are the input variables used to predict the output. Marks is the label because it is the target variable that the machine learning model will predict.

```
[14]
```

```
1s
```

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
X,  
  
y,  
  
test_size=0.2,  
  
random_state=42  
  
)
```

```
print("X_train shape:", X_train.shape)  
  
print("X_test shape:", X_test.shape)  
  
print("y_train shape:", y_train.shape)  
  
print("y_test shape:", y_test.shape)
```

```
X_train shape: (9, 3)  
X_test shape: (3, 3)  
y_train shape: (9,)  
y_test shape: (3,)
```

[15]

0s

```
from sklearn.linear_model import LinearRegression
```

[16]

0s

```
model = LinearRegression()
```

```
model.fit(X_train, y_train)
```

```
[17]
```

```
0s
```

```
print("Model Coefficients (Slope):")
```

```
print(model.coef_)
```

```
print("\nModel Intercept:")
```

```
print(model.intercept_)
```

```
Model Coefficients (Slope):  
[ 3.45487968  1.34491979 -2.40274064]
```

```
Model Intercept:  
-48.100601604278054
```

Question 1: What is the slope?

The slope is represented by the coefficients of the model. It indicates how much the predicted marks change when a feature increases by one unit while keeping other features constant.

Question 2: What is the intercept?

The intercept is the predicted value of marks when all feature values are zero.

Question 3: What do they mean?

The coefficients show the influence of Study_Hours, Attendance, and Assignments on Marks. Positive coefficients indicate that increasing those features generally increases the predicted marks. The intercept provides the baseline prediction.

```
[18]
```

```
0s
```

```
new_students = pd.DataFrame({  
    "Study_Hours": [5, 8, 12],  
    "Attendance": [80, 90, 95],  
    "Assignments": [5, 8, 10]  
})  
  
predictions = model.predict(new_students)  
  
print("Predicted Marks:")  
  
for i, mark in enumerate(predictions, start=1):  
    print(f"Student {i}: {mark:.2f}")
```

```
Predicted Marks:  
Student 1: 64.75  
Student 2: 81.36  
Student 3: 97.10
```

[19]

0s

```
y_pred = model.predict(X_test)
```

[20]

0s

```
comparison = pd.DataFrame({  
    "Actual Marks": y_test.values,  
    "Predicted Marks": y_pred  
})
```

```
comparison["Difference"] = abs(  
    comparison["Actual Marks"] -  
    comparison["Predicted Marks"]  
)
```

```
print(comparison)
```

	Actual Marks	Predicted Marks	Difference
0	92	92.011029	0.011029
1	88	88.269051	0.269051
2	35	31.243984	3.756016

[21]

0s

```
max_error = comparison["Difference"].max()
```

```
student = comparison.loc[  
    comparison["Difference"].idxmax()  
]
```

```
print("\nHighest Prediction Error:")
```

```
print(student)
```

```
Highest Prediction Error:  
Actual Marks      35.000000  
Predicted Marks   31.243984  
Difference         3.756016  
Name: 2, dtype: float64
```

[22]

0s

```
from sklearn.metrics import mean_absolute_error, r2_score
```

[23]

0s

```
mae = mean_absolute_error(y_test, y_pred)
```

```
r2 = r2_score(y_test, y_pred)
```

```
print("Mean Absolute Error (MAE):", mae)
```

```
print("R2 Score:", r2)
```

Mean Absolute Error (MAE): 1.3453654188948245

R² Score: 0.9929962957709836

1. What is MAE?

Mean Absolute Error (MAE) measures the average absolute difference between actual values and predicted values. Lower MAE values indicate better model accuracy.

2. What is R² Score?

R² Score measures how well the model explains the variation in the target variable. A value closer to 1 indicates a better fit.

3. Is your model performing well?

Yes. The model performs well because the MAE is low and the R² Score is close to 1, indicating accurate predictions and a strong fit to the data.[]

[Colab paid products - Cancel contracts here](#)